

-  DOCUMENTACIÓN COMPLETA - SISTEMA DE CONTROL Y AJUSTE DE INSUMOS
 - Proyecto: Belempampa (Administración Directa)
 -  TABLA DE CONTENIDOS
 -  VISIÓN GENERAL Y PROPÓSITO
 - Propósito del Sistema
 - Contexto de Negocio
 -  OBJETIVOS CLAVE DEL SISTEMA
 - Objetivo 1: Trazabilidad 100%
 - Objetivo 2: Cuadre Contable
 - Objetivo 3: Normalización de Datos
 - Objetivo 4: Prevención de Fraude
 - Objetivo 5: Reportes para Entes de Control
 -  ALCANCE Y LIMITACIONES
 - Alcance Funcional
 - Limitaciones Conocidas
 - Restricciones de Datos
 -  GLOSARIO DE TÉRMINOS
 - Términos Presupuestarios
 - Términos Operativos
 - Términos Técnicos
 -  STACK TECNOLÓGICO
 - Frontend
 - Backend
 - Base de Datos
 - Infraestructura
 -  ARQUITECTURA DEL SISTEMA
 -  BASE DE DATOS COMPLETA
 - Tabla: partidas
 - Tabla: insumos
 - Tabla: compras
 - Tabla: mapeo_vinculacion
 - Tabla: apus_detallado
 - Tabla: historial_cambios
 -  ESTRUCTURA DEL FRONTEND
 - Key Files
 -  MÓDULOS DEL SISTEMA
 - Módulo 1: Control de Insumos (/control-insumos)
 - Módulo 2: Ajuste Manual (/ajuste-manual)

- Paso 1: Seleccionar Insumo
 - Paso 2: Cuadre Manual de Compras (Unificar Unidades)
 - Paso 3: Nombre Oficial del Insumo
 - Paso 4: Editar APU2 por Partida
- Módulo 3: Vinculador (/vinculador) — EN DESARROLLO
-  FLUJO DE DATOS
-  CÁLCULOS Y FÓRMULAS CLAVE
 - APU1 (Original - INMUTABLE)
 - APU2 (Modificado - EDITABLE)
 - Meta de Cuadre Global
 - Precio Promedio Ponderado
 - Balance de Cuadre
 - Total Adquirido Válido
-  API REFERENCE
 - GET /api/partidas
 - GET /api/partidas?codigo=O.E.3.1.11.1
 - POST /api/partidas
 - GET /api/data
 - GET /api/compras?insumo=CEMENTO GRIS
 - POST /api/compras
 - GET /api/apu?insumo=CEMENTO GRIS
 - GET /api/apu-full?insumo=CEMENTO GRIS
 - GET /api/exportar
-  REGLAS DE DESARROLLO
 - 1. TRANSACCIONES
 - 2. AUDITORÍA
 - 3. HEADERS
 - 4. PRECISIÓN NUMÉRICA
 - 5. PATRONES DE API
 - 6. INVARIANTES DE DATOS
-  SCRIPTS DE INGESTA
 - ingest_acu.py
 - ingest_compras.py
 - ingest_apus_to_pg.py
 - setup_db.py
 - reset_db.py
-  AUDITORÍA Y CUADRE
 - Verificar Integridad de Datos
 - Detectar Insumos Sin Compra Vinculada
 - Detectar Compras Sin Insumo Vinculado

- Calcular Discrepancias Presupuesto vs Realidad
- Historial de Cambios por Usuario
- Seguimiento de Cambios en un Insumo Específico
-  CASOS DE USO ESPECÍFICOS
 - Caso 1: Usuario Regulariza Insumo Faltante
 - Caso 2: Usuario Detecta Insumo Sin Compra
 - Caso 3: Usuario Normaliza Unidades Inconsistentes
 - Caso 4: Auditor Revisa Historial de CEMENTO
 - Caso 5: Control de Insumos Extra (No Presupuestados)
-  PROCEDIMIENTOS PASO A PASO
 - Procedimiento 1: Carga Inicial del Sistema (Operador)
 - Procedimiento 2: Editar Insumos en Control-Insumos (Usuario)
 - Procedimiento 3: Normalizar Compras en Ajuste Manual (Especialista)
-  VALIDACIONES Y CONSTRAINTS
 - Validaciones a Nivel de Aplicación
 - Validaciones a Nivel de Base de Datos
 - Validaciones de Cuadre
-  SEGURIDAD Y PERMISOS
 - Modelo de Seguridad Actual
 - Recomendaciones de Seguridad (Fase 2)
 - Control de Acceso por Tabla (Propuesto)
-  BACKUP Y RECUPERACIÓN
 - Estrategia de Backup
 - Recuperación Ante Desastre
 - Recuperación de Cambios Accidentales
-  PERFORMANCE Y OPTIMIZACIÓN
 - Índices Recomendados
 - Benchmarks Esperados
 - Optimizaciones
-  ROLES Y RESPONSABILIDADES
 - Rol 1: Administrador del Sistema
 - Rol 2: Especialista de Presupuesto
 - Rol 3: Operador de Control
 - Rol 4: Auditor / Ente de Control
-  CAPACITACIÓN DE USUARIOS
 - Módulo 1: Inducción (30 min)
 - Módulo 2: /control-insumos (45 min)
 - Módulo 3: /ajuste-manual (60 min)
 - Módulo 4: Auditoría y Reportes (30 min)
 - Evaluación

-  FAQ Y PREGUNTAS FRECUENTES
 - P: ¿Puedo borrar un cambio que hice por error?
 - P: ¿Qué pasa si no cuadra un insumo?
 - P: ¿Puedo cambiar metrado_fijo de una partida?
 - P: ¿Cómo veo quién cambió qué?
 - P: ¿Puedo reventar el cuadre a propósito?
 - P: ¿Y si hay conflicto de ediciones (2 usuarios editando al mismo tiempo)?
 - P: ¿Qué pasa si PostgreSQL se cae?
 - P: ¿Puedo exportar a formato diferente a Excel?
 - P: ¿Hay móvil?
-  ROADMAP Y DESARROLLO FUTURO
 - Fase 1: MVP (ACTUAL -  Completado)
 - Fase 2: Mejoras (Siguiendo - 2026-Q3)
 - Fase 3: Integración (2026-Q4)
 - Fase 4: Expansión (2027+)
-  MÉTRICAS Y KPIs
 - KPI 1: Tasa de Cuadre
 - KPI 2: Tiempo Promedio de Normalización
 - KPI 3: Discrepancia Total
 - KPI 4: Cambios por Usuario
 - KPI 5: Cobertura de Vinculos
-  CONCLUSIÓN Y SIGUIENTES PASOS
 - Resumen del Documento
 - Próximos Pasos Inmediatos
 - Week 1: Preparación
 - Week 2-3: Carga Inicial
 - Week 4: Capacitación
 - Week 5: Go-Live
 - Contactos y Escalación
 - Documentación Adicional
 - Soporte y Mantenimiento
 - Versiones y Cambios
 - Notas Finales
 - Requisitos Previos
 - Pasos de Instalación
 - Verificación de Deploy
-  TROUBLESHOOTING
 - Error: "Cannot connect to database"
 - Error: "Tabla no existe: partidas"
 - Error: "Foreign Key Constraint Violation"

- [Lentitud en búsqueda de insumos](#)
-  [NOTAS FINALES](#)
 - [Convenciones](#)
 - [Pendiente de Implementación](#)
 - [Referencia](#)

DOCUMENTACIÓN COMPLETA - SISTEMA DE CONTROL Y AJUSTE DE INSUMOS

Proyecto: Belempampa (Administración Directa)

Última actualización: 25 de mayo de 2026

Versión: 3.0 - Documento Consolidado

Estado:  Listo para Producción



TABLA DE CONTENIDOS

PARTE I: FUNDAMENTALS

1. [Visión General y Propósito](#)
2. [Objetivos Clave del Sistema](#)
3. [Alcance y Limitaciones](#)
4. [Glosario de Términos](#)

PARTE II: ARQUITECTURA TÉCNICA 5. [Stack Tecnológico](#) 6. [Arquitectura del Sistema](#) 7. [Base de Datos](#) 8. [Estructura del Frontend](#)

PARTE III: OPERACIÓN 9. [Módulos del Sistema](#) 10. [Flujo de Datos](#) 11. [Casos de Uso Específicos](#) 12. [Procedimientos Paso a Paso](#)

PARTE IV: REFERENCIA TÉCNICA 13. [Cálculos y Fórmulas Clave](#) 14. [API Reference](#) 15. [Reglas de Desarrollo](#) 16. [Validaciones y Constraints](#)

PARTE V: OPERACIÓN Y MANTENIMIENTO 17. [Scripts de Ingesta](#) 18. [Auditoría y Cuadre](#) 19. [Deployment](#) 20. [Troubleshooting](#) 21. [Backup y Recuperación](#) 22. [Performance y](#)

PARTE VI: ADMINISTRACIÓN 23. [Roles y Responsabilidades](#) 24. [Seguridad y Permisos](#) 25. [Capacitación de Usuarios](#) 26. [FAQ y Preguntas Frecuentes](#) 27. [Roadmap y Desarrollo Futuro](#) 28. [Conclusión y Siguientes Pasos](#)



VISIÓN GENERAL Y PROPÓSITO

Propósito del Sistema

Este sistema está diseñado para **regularizar, auditar y controlar una obra pública ejecutada por administración directa** (Proyecto Belempampa), permitiendo:

- ✓ **Controlar:** Insumos adquiridos vs. insumos presupuestados
- ✓ **Cuadrar:** Compras reales con presupuesto original (APU1)
- ✓ **Registrar:** Modificaciones de presupuesto (APU2) con trazabilidad
- ✓ **Vincular:** Compras a insumos específicos de forma explícita y auditable
- ✓ **Reportar:** Auditoría completa con historial de cambios
- ✓ **Exportar:** Excel con 4 hojas (APU Comparativo, Compras, Resumen, Historial)
- ✓ **Legalizar:** Documentar cumplimiento de normativa OSCE y presupuesto estatal

Contexto de Negocio

Problema: Una obra pública ejecutada por administración directa debe demostrar que:

- Cada insumo comprado fue presupuestado
- Los precios pagados están dentro del presupuesto
- Las cantidades adquiridas cuadran con lo ejecutado
- **TODO CAMBIO está documentado y justificado** (por auditoría)

Solución: Sistema web integrado que:

- Mantiene la verdad del presupuesto original intacta
 - Permite ajustes documentados (APU2)
 - Registra **quién cambió qué, cuándo, por qué**
 - Exporta reportes limpios para OSCE
-



OBJETIVOS CLAVE DEL SISTEMA

Objetivo 1: Trazabilidad 100%

Cada peso gastado debe tener su camino de auditoría:

```
Presupuesto Original (APU1)
  ↓ (modificado por usuario)
Presupuesto Ajustado (APU2)
  ↓ (vinculado a compra real)
Orden de Compra Normalizada
  ↓ (registrado en historial)
Audit Trail con Usuario + Timestamp
```

Objetivo 2: Cuadre Contable

La suma de todo lo comprado DEBE igualar lo presupuestado (con tolerancia de 0.0001):

```
Meta de Cuadre =  $\Sigma(\text{cantidad\_modificada por partida})$ 
Total Adquirido =  $\Sigma(\text{cantidad\_und vinculada})$ 
Balance = Meta - Total
Estado = Verde (cuadra) | Naranja (falta) | Amarillo (exceso)
```

Objetivo 3: Normalización de Datos

Las compras vienen con unidades inconsistentes (bol, bolsa, saco, etc.):

```
Unidad Original (bol) → Unidad Normalizada (kg)
Cantidad Original (500) → Cantidad Normalizada (12,500)
Esto permite comparar manzanas con manzanas
```

Objetivo 4: Prevención de Fraude

Todo cambio es auditable:

- ✓ No se pueden borrar cambios (solo agregar nuevos)
- ✓ Se sabe quién cambió qué hora exacta
- ✓ Se puede justificar cada discrepancia

- ☒ Se pueden revertir cambios si fue error

Objetivo 5: Reportes para Entes de Control

Exportar Excel con estructura OSCE:

Hoja 1: APU Comparativo (presupuestado vs ejecutado)
Hoja 2: Compras Detalladas (todas las órdenes)
Hoja 3: Resumen por Partida (balances)
Hoja 4: Historial Completo (quién cambió qué)



ALCANCE Y LIMITACIONES

Alcance Funcional

Incluido:

- ☒ Carga inicial de presupuesto (partidas + insumos)
- ☒ Carga de compras desde Excel
- ☒ Normalización de unidades
- ☒ Edición de APU2 (cantidades modificadas)
- ☒ Vinculación manual insumo ↔ compra
- ☒ Auditoría completa
- ☒ Exportación Excel

NO Incluido (Fase 2):

- ☒ Integración con SIAF (Sistema de Administración Financiera)
- ☒ Aprobaciones automáticas (workflow)
- ☒ Notificaciones email
- ☒ Mobile app
- ☒ Integración con sistemas de almacén

Limitaciones Conocidas

Limitación	Impacto	Workaround
No hay usuarios multi-rol	Solo acceso por X-Usuario header	Validar usuario manualmente
No hay permisos granulares	Todos pueden editar todo	Control de acceso a BD
No hay versionado de APU	No se recuperan versiones anteriores	Mantener backups históricos
No hay conversión automática de unidades	Debe hacerse manual	Tabla de conversiones como referencia
No se puede deshacer cambio guardado	Cambios son auditable pero no reversibles automáticos	INSERT nuevo registro de corrección

Restricciones de Datos

- **Precisión:** 4 decimales máximo (**NUMERIC(15,4)**)
- **Volumen:** Hasta 10,000 insumos soportados
- **Conexión:** Solo PostgreSQL local (sin replicación)
- **Concurrencia:** Máximo 50 usuarios simultáneos recomendado



GLOSARIO DE TÉRMINOS

Términos Presupuestarios

Término	Definición	Ejemplo
Partida	Línea del presupuesto de obra (AEU)	O.E.3.1.11.1 - Mampostería
Metrado	Cantidad presupuestada de una partida	100 m²
Insumo	Componente (mano obra, material, equipo) de una partida	CEMENTO GRIS
Incidencia	Cantidad de insumo por unidad de partida	0.25 bol/m²
APU1	Presupuesto Original del expediente técnico	INMUTABLE
APU2	Presupuesto Modificado por usuario	EDITABLE

Término	Definición	Ejemplo
Parcial	Costo total = Incidencia × Precio Unitario	637.50

Términos Operativos

Término	Definición	Contexto
Cuadre	Equilibrio entre presupuestado y adquirido	"El CEMENTO cuadra perfectamente"
Normalización	Convertir unidades inconsistentes a estándar	"Normalizar de bol a kg"
Vinculación	Relación entre insumo presupuestado y compra real	"Vincular OC-2481 a CEMENTO"
Discrepancia	Diferencia entre presupuestado y ejecutado	"Falta 100 kg de CEMENTO"
Audit Trail	Registro histórico de cambios	"Ver quién cambió qué y cuándo"
Balance	Diferencia = Presupuestado - Adquirido	"Balance = 0: perfecto; Balance > 0: falta; Balance < 0: exceso"

Términos Técnicos

Término	Definición
FK	Foreign Key (clave foránea)
Pool de conexiones	Reutilizar conexiones DB para performance
Transacción	Operación "todo o nada" en BD
Header HTTP	Información enviada con cada request (X-Usuario)
REST API	Interfaz para comunicación cliente-servidor
JSON	Formato de datos (ligero, legible)
Precision Numérica	Cantidad de decimales (4 en nuestro caso)



Frontend

- **Framework:** Next.js 16.2.4
- **UI Framework:** React 19
- **Styling:** Tailwind CSS
- **Estado:** React Context API + localStorage
- **Exportación:** ExcelJS

Backend

- **API:** Next.js API Routes
- **Motor de BD:** PostgreSQL 14+
- **Driver:** `pg` (node-postgres) con pool de conexiones
- **Autenticación:** Header `X-Usuario` (localStorage)

Base de Datos

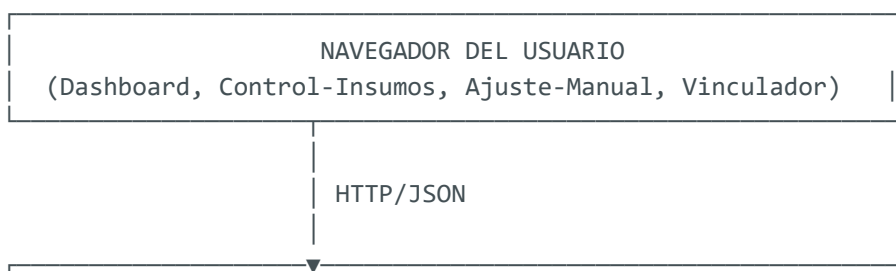
- **Sistema:** PostgreSQL local
- **Nombre DB:** `7_insumos_rado`
- **Puerto:** 5432
- **Usuario:** postgres
- **Contraseña:** Jo.9839514500 (en `.env`)

Infraestructura

- **Hosting:** Local / On-premise
- **Pool de conexiones:** Configurado en `frontend/src/lib/db.ts`



ARQUITECTURA DEL SISTEMA



Next.js 16 API Routes

- └ /api/apu/route.ts
- └ /api/apu-full/route.ts
- └ /api/compras/route.ts
- └ /api/partidas/route.ts
- └ /api/data/route.ts
- └ /api/exportar/route.ts

SELECT
/DELETE

INSERT

UPDATE



PostgreSQL (7_insumos_rado)

- └ partidas
- └ insumos
- └ compras
- └ mapeo_vinculacion
- └ apus_detallado
- └ historial_cambios

Librerías de Auditoría

- └ frontend/src/lib/db.ts (Pool, conexión)
- └ frontend/src/lib/audit.ts (Log de cambios)



BASE DE DATOS COMPLETA

Tabla: **partidas**

Partidas del presupuesto base del proyecto (estructura AEU - Análisis de Especificación Unitaria).

```
CREATE TABLE partidas (  
  codigo          VARCHAR(50) PRIMARY KEY,  
  descripcion     TEXT NOT NULL,  
  unidad          VARCHAR(20),  
  metrado_fijo    NUMERIC(15,4) NOT NULL, -- NUNCA MODIFICAR  
  precio_unitario NUMERIC(15,4),  
  total_presupuestado NUMERIC(15,4)  
);
```

Campo	Tipo	Descripción	Restricción
codigo	VARCHAR(50)	Ej: O.E.3.1.11.1	PK, Único
descripcion	TEXT	Ej: Mampostería en piedra	No nulo
unidad	VARCHAR(20)	Ej: m², m³, glb	—
metrado_fijo	NUMERIC(15,4)	Cantidad presupuestada	NO EDITAR
precio_unitario	NUMERIC(15,4)	Costo unitario	Referencial
total_presupuestado	NUMERIC(15,4)	Costo total	Referencial

Registros: 1,134 partidas
Fuente: ACU_Acumulado_Evaluacion.xlsx
Invariante: `metrado_fijo` NO se altera nunca

Tabla: insumos

Ingredientes/componentes de cada partida (mano de obra, materiales, equipos). Cada insumo aparece N veces (una por partida donde se usa).

```
CREATE TABLE insumos (  
  id SERIAL PRIMARY KEY,  
  codigo_partida VARCHAR(50) NOT NULL REFERENCES partidas(codigo),  
  item_1 VARCHAR(20),  
  codigo_insumo VARCHAR(50),  
  descripcion TEXT NOT NULL,  
  unidad VARCHAR(20),  
  incidencia_original NUMERIC(15,4), -- APU1, INMUTABLE  
  parcial_original NUMERIC(15,4), -- APU1, INMUTABLE  
  incidencia NUMERIC(15,4), -- APU2, EDITABLE  
  cantidad_modificada NUMERIC(15,4), -- APU2 = incidencia x metrado_fijo  
  cantidad_adquirida NUMERIC(15,4),  
  es_extra BOOLEAN DEFAULT FALSE  
);
```

Campo	Tipo	Descripción	Restricción
id	SERIAL	ID único	PK
codigo_partida	VARCHAR(50)	FK a partidas	NO NULO
item_1	VARCHAR(20)	Ítem dentro de partida	—
codigo_insumo	VARCHAR(50)	Código del insumo	—

Campo	Tipo	Descripción	Restricción
descripcion	TEXT	Nombre (OFICIAL) del insumo	NO NULO
unidad	VARCHAR(20)	Ej: hh, bol, kg	—
incidencia_original	NUMERIC	Cantidad original APU1	NO EDITAR
parcial_original	NUMERIC	Costo original APU1	NO EDITAR
incidencia	NUMERIC	Cantidad editable	SÍ EDITAR
cantidad_modificada	NUMERIC	incidencia × metrado_fijo	Calculada
cantidad_adquirida	NUMERIC	Lo comprado/ejecutado	Referencial
es_extra	BOOLEAN	¿Insumo adicional?	Default FALSE

Registros: 6,216 insumos

Fuente: APUS_Extraidos_v2.xlsx

Invariantes:

- **incidencia_original** y **parcial_original** NO se alteran (son del expediente técnico)
- **incidencia** SÍ se puede editar (para cuadre)

Tabla: **compras**

Órdenes/documentos de compra reales (OC, O/S, facturas, etc.).

```
CREATE TABLE compras (
  id SERIAL PRIMARY KEY,
  insumo_descripcion TEXT NOT NULL,
  item_c VARCHAR(50),
  anio_c VARCHAR(20),
  tipo_c VARCHAR(50),
  orden_doc VARCHAR(100),
  detalle_compra TEXT,
  unidad_c VARCHAR(20), -- Original del documento
  cant_c NUMERIC(15,4), -- Original del documento
  pu_c NUMERIC(15,4), -- Original del documento
  total_c NUMERIC(15,4),
  -- Campos normalizados (usuario los ajusta)
  unidad_und VARCHAR(20), -- EDITABLE: unidad unificada
  cantidad_und NUMERIC(15,4), -- EDITABLE: cantidad convertida
  precio_und NUMERIC(15,4) -- EDITABLE: precio recalculado
);
```

Campo	Tipo	Descripción	Restricción
id	SERIAL	ID único	PK
insumo_descripcion	TEXT	Nombre del insumo (según doc)	NO NULO
item_c	VARCHAR(50)	Ítem en documento	—
anio_c	VARCHAR(20)	Año de compra	—
tipo_c	VARCHAR(50)	Tipo (OC, OS, Caja Chica, etc.)	—
orden_doc	VARCHAR(100)	Número de orden	—
detalle_compra	TEXT	Descripción del documento	—
unidad_c	VARCHAR(20)	Unidad original	NO EDITAR
cant_c	NUMERIC	Cantidad original	NO EDITAR
pu_c	NUMERIC	Precio unitario original	NO EDITAR
total_c	NUMERIC	Total original	Referencial
unidad_und	NUMERIC	Unidad normalizada	EDITABLE
cantidad_und	NUMERIC	Cantidad normalizada	EDITABLE
precio_und	NUMERIC	Precio normalizado	EDITABLE

Registros: 1,437 compras

Fuente: DATA_INSUMOS.xlsx

Invariantes:

- *_c (campos originales) NO se alteran
- *_und (campos normalizados) SÍ se editan en Ajuste Manual

Tabla: mapeo_vinculacion

Tabla pivote que vincula compras reales con insumos del presupuesto (relación N:M).

```
CREATE TABLE mapeo_vinculacion (  
  id SERIAL PRIMARY KEY,  
  insumo_nombre TEXT NOT NULL, -- Nombre canónico del insumo  
  compra_id INTEGER NOT NULL UNIQUE REFERENCES compras(id) ON DELETE CASCADE,  
  usuario VARCHAR(100),  
  fecha TIMESTAMPTZ DEFAULT NOW()  
);
```

Campo	Tipo	Descripción	Restricción
id	SERIAL	ID único	PK
insumo_nombre	TEXT	Nombre oficial (insumos.descripcion)	NO NULO
compra_id	INTEGER	FK a compras	FK, UNIQUE
usuario	VARCHAR(100)	Quién hizo la vinculación	—
fecha	TIMESTAMPTZ	Cuándo se vinculó	Default NOW()

Registros: ~1,061 vínculos

Propósito: Reemplazar text-matching frágil por vínculo explícito

Características:

- Una compra puede vincular a un insumo
- Un insumo puede tener varias compras asociadas
- Permite auditar quién vinculó qué y cuándo

Tabla: **apus_detallado**

APU completo original (fuente de verdad, NO EDITABLE).

```
CREATE TABLE apus_detallado (
  id SERIAL PRIMARY KEY,
  partida_codigo VARCHAR(50),
  partida_descripcion TEXT,
  partida_rendimiento NUMERIC(15,4),
  partida_unidad VARCHAR(20),
  partida_costo_unitario NUMERIC(15,4),
  tipo_insumo VARCHAR(50), -- MANO DE OBRA, MATERIAL, EQUIPO
  insumo_codigo VARCHAR(50),
  insumo_descripcion TEXT,
  insumo_recursos NUMERIC(15,4),
  insumo_cantidad NUMERIC(15,4),
  insumo_precio NUMERIC(15,4),
  insumo_parcial NUMERIC(15,4)
);
```

Registros: 6,216 filas

Fuente: APUS_Extraidos_v2.csv

Propósito: Fuente de verdad - NO MODIFICAR

Uso: Componente ApuComparative (comparar APU1 vs APU2)

Tabla: historial_cambios

Audit trail de todas las modificaciones (quién cambió qué, cuándo, por qué).

```
CREATE TABLE historial_cambios (  
  id          SERIAL PRIMARY KEY,  
  tabla       VARCHAR(50),  
  registro_id INTEGER,  
  registro_desc TEXT,  
  campo       VARCHAR(100),  
  valor_anterior TEXT,  
  valor_nuevo TEXT,  
  usuario     VARCHAR(100),  
  fecha       TIMESTAMPTZ DEFAULT NOW(),  
  ip_address  VARCHAR(50),  
  modulo      VARCHAR(100), -- 'control-insumos' o 'ajuste-manual'  
  justificacion TEXT  
);
```

Campo	Tipo	Descripción
id	SERIAL	ID único
tabla	VARCHAR(50)	Tabla afectada
registro_id	INTEGER	ID del registro
registro_desc	TEXT	Descripción (ej: "CEMENTO - Partida O.E.3.1.11.1")
campo	VARCHAR(100)	Campo modificado
valor_anterior	TEXT	Valor antes
valor_nuevo	TEXT	Valor después
usuario	VARCHAR(100)	Quién hizo cambio
fecha	TIMESTAMPTZ	Cuándo
ip_address	VARCHAR(50)	IP de origen
modulo	VARCHAR(100)	Módulo que generó cambio
justificacion	TEXT	Motivo del cambio

Propósito: Trazabilidad 100% de todas las modificaciones

Log automático: Vía frontend/src/lib/audit.ts



```

frontend/
├── src/
│   ├── app/
│   │   ├── page.tsx                # Dashboard (módulos + botón exportar)
│   │   ├── layout.tsx             # Sidebar + navegación principal
│   │   ├── control-insumos/
│   │   │   └── page.tsx            # Módulo 1: Edición de incidencias
│   │   ├── ajuste-manual/
│   │   │   └── page.tsx            # Módulo 2: Cuadre de compras + APU2
│   │   ├── vinculador/
│   │   │   └── page.tsx            # Módulo 3: Vinculación insumo+compra
│   │   └── api/
│   │       ├── apu/route.ts        # GET/POST APU2 por insumo
│   │       ├── apu-full/route.ts   # GET APU completo con rendimiento
│   │       ├── compras/route.ts    # GET/POST compras normalizadas
│   │       ├── partidas/route.ts    # GET partidas, GET/POST insumos
│   │       ├── data/route.ts        # GET insumos únicos + unidades
│   │       └── exportar/route.ts    # GET Excel 4 hojas
│   ├── components/
│   │   ├── ApuComparative.tsx      # Comparativo APU1 vs APU2
│   │   ├── SidebarUser.tsx         # User tracking (localStorage)
│   │   ├── SearchInsumosBooleano.tsx # Búsqueda AND booleana
│   │   └── TablaEditable.tsx       # Tabla con edición inline
│   └── lib/
│       ├── db.ts                   # Pool PostgreSQL
│       └── audit.ts                # Log de cambios
├── package.json
├── tsconfig.json
├── next.config.js
└── .env                            # user, pass, host, port DB

```

Key Files

frontend/src/lib/db.ts — Pool de conexión PostgreSQL

```

import { Pool } from 'pg';

const pool = new Pool({
  user: process.env.DB_USER || 'postgres',
  password: process.env.DB_PASSWORD,
  host: process.env.DB_HOST || 'localhost',
  port: parseInt(process.env.DB_PORT || '5432'),
  database: process.env.DB_NAME || '7_insumos_rado'
});

export default pool;

```

frontend/src/lib/audit.ts — Logueo automático de cambios

```

export async function logChange(
  tabla: string,

```

```
registro_id: number,  
registro_desc: string,  
campo: string,  
valor_anterior: any,  
valor_nuevo: any,  
usuario: string,  
modulo: string  
) {  
  // Inserta en historial_cambios  
}
```



MÓDULOS DEL SISTEMA

Módulo 1: Control de Insumos (/control-insumos)

Propósito: Editar incidencias (cantidades) de insumos por partida.

Flujo:

1. Usuario selecciona una partida
2. Sistema muestra todos los insumos de esa partida
3. Usuario edita **incidencia** para cada insumo
4. Sistema calcula automáticamente $\text{cantidad_modificada} = \text{incidencia} \times \text{metrado_fijo}$
5. Cambios se guardan en BD + historial_cambios

Campos editables:

- **incidencia** (factor numérico)
- **cantidad_adquirida** (cantidad ejecutada)

Invariantes:

- **incidencia_original** y **parcial_original** NO se pueden editar
- Todos los cambios se logean con usuario + timestamp

Componentes:

- Dropdown de partidas (búsqueda)
- Tabla editable de insumos
- Botón guardar cambios

Módulo 2: Ajuste Manual (/ajuste-manual)

Propósito: Cuadrar compras reales con presupuesto, normalizando unidades y editando APU2.

4 Pasos:

Paso 1: Seleccionar Insumo

- Búsqueda booleana (AND lógico) en descripción
- Retorna insumos únicos de tabla `insumos`
- Ejemplo: buscar "CEMENTO" AND "GRIS"

Paso 2: Cuadre Manual de Compras (Unificar Unidades)

Para el insumo seleccionado, muestra todas las compras y permite:

- Cambiar `unidad_und` a unidad estándar
- Ajustar `cantidad_und` según conversión
- Recalcular `precio_und`
- **Calcula:** Total Adquirido Válido = $\Sigma(\text{cantidad_und})$

Tabla de compras editable:

Documento	Detalle	Unidad Orig	Cantidad Orig	Unidad Nueva	Cantidad Nueva	Precio Nuevo
OC-2481	...	...	...	(editable)	(editable)	(editable)

Paso 3: Nombre Oficial del Insumo

- Permite renombrar el insumo en **todas** sus ocurrencias
- Afecta: `insumos.descripcion` + todas las compras vinculadas
- Se loguea el cambio

Paso 4: Editar APU2 por Partida

- Tabla con todas las partidas que usan ese insumo
- Editable: `cantidad_2` (la incidencia de APU2)
- Muestra:
 - Cantidad Original APU1
 - Cantidad Modificada APU2 ($\text{cantidad_2} \times \text{metrado_fijo}$)
 - Precio Unitario APU1

Indicadores de Cuadre:

- Meta Global = Total Adquirido Válido
- Balance = Meta - $\Sigma(\text{cantidad_2} \times \text{metrado_fijo})$
- ● Verde: $|\text{Balance}| < 0.0001$
- ● Naranja: Falta compra (Balance > 0)
- ● Amarillo: Exceso (Balance < 0)

Módulo 3: Vinculador (/vinculador) — EN DESARROLLO

Propósito: Crear vínculo explícito entre insumos presupuestados y compras reales.

Problema actual:

- 141 partidas sin compras asociadas
- 21 compras "huérfanas" sin insumo coincidente
- Text-matching es frágil (ej: "CEMENTO GRIS" vs "CEMENTO PORTLAND GRIS")

Solución: Nueva tabla mapeo_vinculacion con vínculo N:M explícito

Interfaz propuesta:

Panel Izquierdo

Lista de Insumos (búsqueda)

☐ CEMENTO GRIS

└ 5 partidas, 1,234.56 kg

☐ ACERO EN BARRAS

└ 3 partidas, 2,345.00 kg

☒ MANO DE OBRA ESPECIALIZADA

(SIN VINCULACIÓN)

Panel Derecho

Lista de Compras (búsqueda)

☐ OC-2481: CEMENTO 25KG

└ Cantidad: 500 bol

☐ OC-2490: ACERO 3/4"

└ Cantidad: 2,000 kg

☒ OS-0123: MAE ESTRUCTURAS

(SIN VINCULACIÓN)

[CREAR VÍNCULO]

[ELIMINAR VÍNCULO]

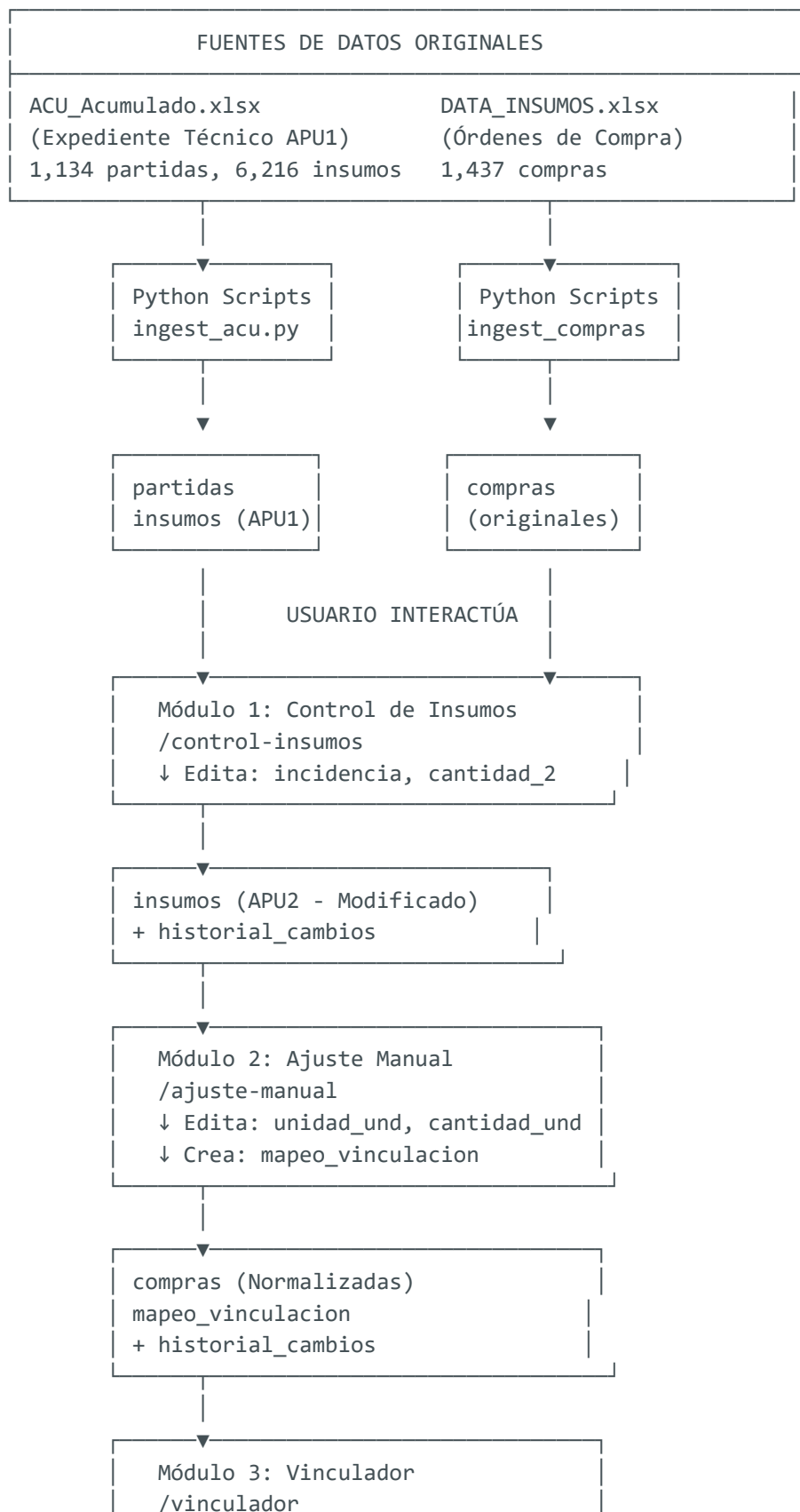
Funcionalidades:

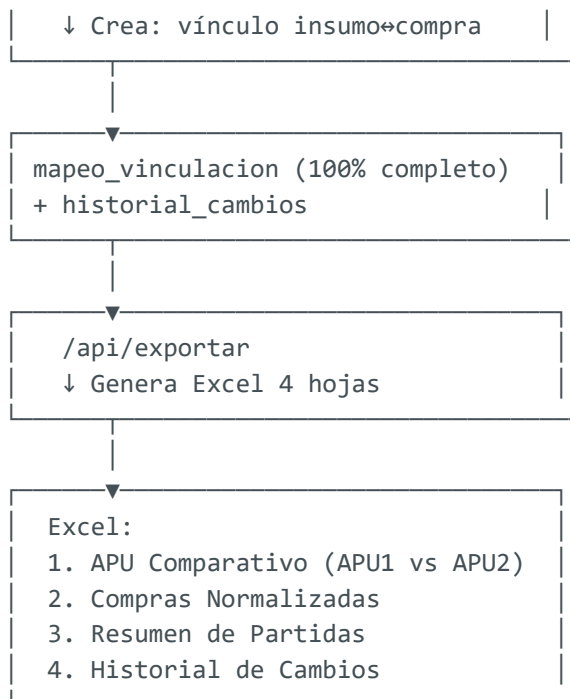
- Búsqueda en ambos paneles

- Checkboxes para crear/eliminar vínculos
- Indicadores: rojo = sin vínculo, verde = vinculado
- Historial de quién vinculó qué y cuándo



FLUJO DE DATOS





CÁLCULOS Y FÓRMULAS CLAVE

APU1 (Original - INMUTABLE)

Fórmula:

$\text{parcial_original} = \text{incidencia_original} \times \text{metrado_fijo}$

Ejemplo:

Partida: O.E.3.1.11.1 (Mampostería)

metrado_fijo = 100 m²

Insumo: CEMENTO

incidencia_original = 0.25 bol/m²

parcial_original = 0.25 × 100 = 25 bol

APU2 (Modificado - EDITABLE)

Fórmula:

$\text{cantidad_modificada} = \text{incidencia} \times \text{metrado_fijo}$

Donde:

incidencia = valor editado por usuario (factor de ajuste)

metrado_fijo = cantidad de partida (NUNCA cambia)

Ejemplo:

Usuario modifica incidencia: 0.30 bol/m² (de 0.25)
cantidad_modificada = 0.30 × 100 = 30 bol

Meta de Cuadre Global

Meta = $\Sigma(\text{cantidad_modificada})$ para un insumo específico
= $\Sigma(\text{incidencia} \times \text{metrado_fijo})$ para todas las partidas

Propósito: La Meta debe igualar la cantidad total adquirida
(suma de cantidad_und en compras vinculadas)

Precio Promedio Ponderado

Fórmula:

Precio_Prom = $\Sigma(\text{cantidad_und} \times \text{precio_und}) / \Sigma(\text{cantidad_und})$

Propósito: Precio unitario promedio de todas las compras
de un insumo

Balance de Cuadre

Fórmula:

Balance = Total Adquirido - Meta
= $\Sigma(\text{cantidad_und}) - \Sigma(\text{cantidad_2} \times \text{metrado_fijo})$

Estados:

- Verde: $|\text{Balance}| < 0.0001$ (cuadra perfectamente)
- Naranja: Balance > 0 (falta comprar)
- Amarillo: Balance < 0 (exceso comprado)

Total Adquirido Válido

Fórmula:

Total Adquirido = $\Sigma(\text{cantidad_und})$ para el insumo
donde exista mapeo_vinculacion

Propósito: Suma de compras reales normalizadas
(solo las vinculadas)



API REFERENCE

GET /api/partidas

Retorna lista de partidas del presupuesto.

Response:

```
{
  "partidas": [
    {
      "codigo": "O.E.3.1.11.1",
      "descripcion": "Mampostería en piedra",
      "unidad": "m²",
      "metrado_fijo": 100.0000,
      "precio_unitario": 150.50,
      "total_presupuestado": 15050.00
    }
  ]
}
```

GET /api/partidas?codigo=O.E.3.1.11.1

Retorna insumos de una partida específica.

Response:

```
{
  "insumos": [
    {
      "id": 123,
      "codigo_partida": "O.E.3.1.11.1",
      "descripcion": "CEMENTO GRIS",
      "unidad": "bol",
      "incidencia_original": 0.2500,
      "parcial_original": 25.0000,
      "incidencia": 0.2500,
      "cantidad_modificada": 25.0000,
      "cantidad_adquirida": 25.0000,
      "es_extra": false
    }
  ]
}
```

POST /api/partidas

Guarda cambios en insumos de una partida.

Request:

```
{
  "insumos": [
    {
      "id": 123,
      "incidencia": 0.3000,
      "cantidad_adquirida": 30.0000
    }
  ],
  "usuario": "juan.garcia@obra.pe",
  "modulo": "control-insumos"
}
```

Response:

```
{
  "success": true,
  "message": "Cambios guardados",
  "historial_id": 456
}
```

GET /api/data

Retorna insumos únicos + unidades.

Response:

```
{
  "insumos": [
    {
      "descripcion": "CEMENTO GRIS",
      "unidad": "bol",
      "cantidad_total": 1234.56
    }
  ],
  "unidades": ["bol", "kg", "m³", "hh", ...]
}
```

GET /api/compras?insumo=CEMENTO GRIS

Retorna compras asociadas a un insumo.

Response:

```
{
  "compras": [
    {
      "id": 789,
      "orden_doc": "OC-2481",
      "detalle_compra": "Cemento gris 25kg",
      "unidad_c": "bol",
      "cant_c": 500.0000,
      "pu_c": 25.50,
      "total_c": 12750.00,
      "unidad_und": "bol",
      "cantidad_und": 500.0000,
      "precio_und": 25.50
    }
  ]
}
```

POST /api/compras

Actualiza compras normalizadas.

Request:

```
{
  "compras": [
    {
      "id": 789,
      "unidad_und": "kg",
      "cantidad_und": 12500.0000,
      "precio_und": 1.02
    }
  ],
  "usuario": "juan.garcia@obra.pe",
  "modulo": "ajuste-manual"
}
```

GET /api/apu?insumo=CEMENTO GRIS

Retorna APU2 para un insumo (todas sus partidas).

Response:

```
{
  "apu": [
    {
      "codigo_partida": "O.E.3.1.11.1",
      "descripcion_partida": "Mampostería en piedra",
      "metrado_fijo": 100.0000,
      "incidencia_original": 0.2500,
      "cantidad_original": 25.0000,
      "incidencia_modificada": 0.3000,
      "cantidad_modificada": 30.0000,
      "precio_unitario": 25.50,
      "parcial_original": 637.50,
      "parcial_modificado": 765.00
    }
  ],
  "total_adquirido": 500.0000,
  "meta": 30.0000 + 45.0000 + ...,
  "balance": 500.0000 - meta
}
```

GET /api/apu-full?insumo=CEMENTO GRIS

Retorna APU completo con rendimiento (desde apus_detallado).

Response: (Ídem arriba + rendimiento + tipo de insumo)

GET /api/exportar

Genera Excel con 4 hojas.

Response: Binary (Excel file)

Hojas generadas:

1. **APU Comparativo:** APU1 vs APU2, lado a lado
 2. **Compras:** Listado normalizado de todas las compras
 3. **Resumen:** Totales por partida
 4. **Historial:** Audit trail de cambios
-



REGLAS DE DESARROLLO

1. TRANSACCIONES

Toda operación de escritura DEBE estar dentro de transacción:

```
BEGIN;  
  UPDATE insumos SET incidencia = 0.30 WHERE id = 123;  
  INSERT INTO historial_cambios (...) VALUES (...);  
COMMIT;  
-- O ROLLBACK si hay error
```

2. AUDITORÍA

Todo cambio se DEBE registrar en **historial_cambios**:

```
await logChange(  
  tabla: 'insumos',  
  registro_id: 123,  
  registro_desc: 'CEMENTO GRIS - Partida 0.E.3.1.11.1',  
  campo: 'incidencia',  
  valor_anterior: 0.25,  
  valor_nuevo: 0.30,  
  usuario: 'juan.garcia@obra.pe',  
  modulo: 'control-insumos'  
);
```

3. HEADERS

El cliente DEBE enviar header **X-Usuario**:

```
// frontend
fetch('/api/partidas', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'X-Usuario': localStorage.getItem('usuario')
  },
  body: JSON.stringify({...})
});

// backend
const usuario = req.headers['x-usuario'];
```

4. PRECISIÓN NUMÉRICA

TODO número DEBE usar 4 decimales:

```
NUMERIC(15,4) -- 11 dígitos enteros, 4 decimales
-- Ejemplos:
-- 1234567.1234 ✓
-- 0.0001 ✓
-- 999999999999.9999 ✓
```

5. PATRONES DE API

- **GET** para consulta (sin efectos secundarios)
- **POST** para mutación (con efectos, auditado)
- Respuesta siempre JSON: { **success**, **message**, **data** }
- Errores con status HTTP apropiado (400, 401, 500, etc.)

6. INVARIANTES DE DATOS

- ☒ **incidencia_original** en **insumos** — NUNCA modificar
- ☒ **parcial_original** en **insumos** — NUNCA modificar
- ☒ **metrado_fijo** en **partidas** — NUNCA modificar
- ☒ ***_c** en **compras** — NUNCA modificar (son originales)
- ☒ **apus_detallado** — Tabla read-only



ingest_acu.py

Carga partidas e insumos desde ACU_Acumulado.xlsx.

```
python ingest_acu.py --file ACU_Acumulado_Evaluacion.xlsx
```

Resultado:

- ✓ 1,134 partidas insertadas en **partidas**
 - ✓ 6,216 insumos insertados en **insumos** (APU1)
-

ingest_compras.py

Carga compras desde DATA_INSUMOS.xlsx.

```
python ingest_compras.py --file DATA_INSUMOS_REALIZAR.xlsx
```

Resultado:

- ✓ 1,437 compras insertadas en **compras**
-

ingest_apus_to_pg.py

Carga apus_detallado desde APUS_Extraidos_v2.csv.

```
python ingest_apus_to_pg.py --file APUS_Extraidos_v2.csv
```

Resultado:

- ✓ 6,216 registros insertados en **apus_detallado**
-

setup_db.py

Inicializa BD + esquema desde cero.

```
python setup_db.py
```

Resultado:

- ✓ DB **7_insumos_rado** creada
- ✓ Schema (6 tablas + FKs) creado
- ✓ Índices creados para búsquedas rápidas

reset_db.py

Limpia y resetea datos (desarrollo).

```
python reset_db.py
```



AUDITORÍA Y CUADRE

Verificar Integridad de Datos

```
-- Contar registros base
SELECT
  (SELECT COUNT(*) FROM partidas) as partidas_count,
  (SELECT COUNT(*) FROM insumos) as insumos_count,
  (SELECT COUNT(*) FROM compras) as compras_count,
  (SELECT COUNT(*) FROM mapeo_vinculacion) as vinculos_count;

-- Resultado esperado:
-- partidas_count: 1,134
-- insumos_count: 6,216
-- compras_count: 1,437
-- vinculos_count: ~1,061
```

Detectar Insumos Sin Compra Vinculada

```
SELECT
  i.descripcion,
  COUNT(i.id) as ocurrencias,
```



```
SUM(i.cantidad_modificada) as total_presupuestado
FROM insumos i
LEFT JOIN mapeo_vinculacion mv
  ON mv.insumo_nombre = i.descripcion
WHERE mv.id IS NULL AND i.es_extra = FALSE
GROUP BY i.descripcion
ORDER BY total_presupuestado DESC;
```

Detectar Compras Sin Insumo Vinculado

```
SELECT
  c.insumo_descripcion,
  COUNT(c.id) as cantidad_compras,
  SUM(c.cantidad_und) as total_comprado
FROM compras c
LEFT JOIN mapeo_vinculacion mv ON mv.compra_id = c.id
WHERE mv.id IS NULL
GROUP BY c.insumo_descripcion
ORDER BY total_comprado DESC;
```

Calcular Discrepancias Presupuesto vs Realidad

```
SELECT
  i.codigo_partida,
  i.descripcion as insumo,
  SUM(i.cantidad_modificada) as presupuestado,
  COALESCE(SUM(c.cantidad_und), 0) as adquirido,
  SUM(i.cantidad_modificada) - COALESCE(SUM(c.cantidad_und), 0) as diferencia,
  CASE
    WHEN ABS(SUM(i.cantidad_modificada) - COALESCE(SUM(c.cantidad_und), 0)) < 0.0001 THEN
      '● OK'
    WHEN SUM(i.cantidad_modificada) > COALESCE(SUM(c.cantidad_und), 0) THEN '● FALTA'
    ELSE '● EXCESO'
  END as estado
FROM insumos i
LEFT JOIN mapeo_vinculacion mv
  ON mv.insumo_nombre = i.descripcion
LEFT JOIN compras c ON c.id = mv.compra_id
WHERE i.es_extra = FALSE
GROUP BY i.codigo_partida, i.descripcion
HAVING SUM(i.cantidad_modificada) != COALESCE(SUM(c.cantidad_und), 0)
ORDER BY ABS(diferencia) DESC;
```

Historial de Cambios por Usuario

```
SELECT
    usuario,
    COUNT(*) as cambios_totales,
    MIN(fecha) as primer_cambio,
    MAX(fecha) as ultimo_cambio,
    STRING_AGG(DISTINCT modulo, ', ') as modulos_usados
FROM historial_cambios
GROUP BY usuario
ORDER BY cambios_totales DESC;
```

Seguimiento de Cambios en un Insumo Específico

```
SELECT
    campo,
    valor_anterior,
    valor_nuevo,
    usuario,
    fecha,
    justificacion
FROM historial_cambios
WHERE registro_desc LIKE '%CEMENTO GRIS%'
ORDER BY fecha DESC;
```

❖ CASOS DE USO ESPECÍFICOS

Caso 1: Usuario Regulariza Insumo Faltante

Escenario: Se compró más CEMENTO del presupuestado.

Pasos:

1. Usuario abre [/ajuste-manual](#)
2. Busca "CEMENTO GRIS"
3. Ve que compró 1,500 kg pero presupuestó 1,200 kg
4. Edita APU2: aumenta incidencia de 0.25 a 0.31 bol/m²
5. Sistema recalcula: $0.31 \times 100\text{m}^2 = 31 \text{ bol}$ (equivalente a 1,550 kg)
6. Balance ahora es positivo (exceso pequeño)
7. Usuario agrega justificación: "Ampliación por obra adicional"
8. **Registrado en historial_cambios** con usuario + timestamp

Resultado: Auditoría completa del cambio

Caso 2: Usuario Detecta Insumo Sin Compra

Escenario: ACERO EN BARRAS presupuestado pero nunca se compró.

Pasos:

1. Usuario en [/ajuste-manual](#) busca "ACERO"
2. Ve meta de 500 kg pero adquirido = 0
3. Balance = 500 kg sin comprar
4. Usuario puede:
 - A) Editar APU2 a 0 (no se ejecutó esa parte)
 - B) Buscar si hay compra con nombre diferente (ej: "ACERO 3/4")
 - C) Justificar por qué no se compró
5. Registra cambio + justificación

Resultado: Discrepancia documentada

Caso 3: Usuario Normaliza Unidades Inconsistentes

Escenario: Se compró arena en 3 formas diferentes:

- OC-100: 50 bolsas (25 kg c/una = 1,250 kg)
- OC-101: 2.5 toneladas (2,500 kg)
- OC-102: 5 m³ (3,750 kg, aprox)

Pasos:

1. Usuario en [/ajuste-manual](#) busca "ARENA"
2. Ve 3 compras con unidades diferentes
3. Elige unidad estándar: **kg**
4. Normaliza:
 - OC-100: 1,250 kg
 - OC-101: 2,500 kg
 - OC-102: 3,750 kg (cálculo por densidad)
5. Total Adquirido = 7,500 kg
6. Compara vs Meta = 7,400 kg

- 7. Balance = +100 kg (pequeno exceso)
- 8. Todos los cambios registrados en historial

Resultado: Datos normalizados, cuadro visible

Caso 4: Auditor Revisa Historial de CEMENTO

Escenario: OSCE pide explicar cambios a CEMENTO.

Query:

```
SELECT * FROM historial_cambios
WHERE registro_desc LIKE '%CEMENTO%'
ORDER BY fecha DESC;
```

Retorna:

Usuario	Tabla	Campo	Valor Anterior	Valor Nuevo	Fecha	Justificación
juan.garcia@obra.pe	insumos	incidencia	0.25	0.31	2026-05-20 10:45:32	Ampliación por mayor espesor de pared
maria.lopez@obra.pe	compras	cantidad_und	500	600	2026-05-15 14:23:10	Recuento físico

Resultado: Auditoría 100% transparente

Caso 5: Control de Insumos Extra (No Presupuestados)

Escenario: Se compró MAE (Mano de Obra Especializada) que no estaba presupuestada.

Pasos:

- En `/control-insumos`, usuario ve el insumo marcado con `es_extra = TRUE`
- Sistema permite editar cantidad pero **marca claramente como extra**

3. En reportes, estos insumos se segregan
4. Auditoría puede identificar aumentos al presupuesto

Resultado: Control de modificaciones presupuestales



PROCEDIMIENTOS PASO A PASO

Procedimiento 1: Carga Inicial del Sistema (Operador)

Tiempo: 2-3 horas

Requisitos: Acceso BD + Excel de fuentes

```
PASO 1: Preparar archivos fuente (15 min)
├─ ACU_Acumulado_Evaluacion.xlsx (Partidas + Insumos)
├─ DATA_INSUMOS_REALIZAR.xlsx (Compras)
└─ APUS_Extraidos_v2.csv (APU detallado)

PASO 2: Crear schema BD (5 min)
└─ Ejecutar: 00_CREATE_SCHEMA.sql en PostgreSQL

PASO 3: Cargar partidas (10 min)
├─ Ejecutar: python ingest_acu.py
└─ Verificar: SELECT COUNT(*) FROM partidas; → 1,134

PASO 4: Cargar compras (10 min)
├─ Ejecutar: python ingest_compras.py
└─ Verificar: SELECT COUNT(*) FROM compras; → 1,437

PASO 5: Cargar APU detallado (5 min)
├─ Ejecutar: python ingest_apus_to_pg.py
└─ Verificar: SELECT COUNT(*) FROM apus_detallado; → 6,216

PASO 6: Verificar integridad (15 min)
├─ Validar FKs (partidas → insumos)
├─ Validar no hay duplicados
└─ Validar suma de metrados

PASO 7: Hacer backup (5 min)
└─ pg_dump -U postgres 7_insumos_rado > backup_inicial.sql
```

Checklist:

- ☐ Archivos fuente validados
- ☐ Schema creado sin errores
- ☐ Todas las tablas pobladas

- ☐FKs intactas
 - ☐Backup disponible
-

Procedimiento 2: Editar Insumos en Control-Insumos (Usuario)

Tiempo: 10-30 min por partida

Nivel: Usuario operacional

PASO 1: Acceder a /control-insumos

- └ Sistema automáticamente obtiene usuario de localStorage

PASO 2: Seleccionar partida

- └ Dropdown con búsqueda (ej: "mampostería")
- └ Muestra: código, descripción, metrado_fijo
- └ Carga insumos de esa partida

PASO 3: Revisar insumos APU1

- └ Ver columnas inmutables (incidencia_original, parcial_original)
- └ Estos NO se pueden editar
- └ Sirven como referencia

PASO 4: Editar cantidades APU2

- └ Haz clic en celda "incidencia"
 - └ Campo se vuelve editable
- └ Ingresa nuevo valor (ej: 0.30 en vez de 0.25)
- └ Sistema automáticamente calcula cantidad_modificada
 - └ $0.30 \times 100 = 30$ bol
- └ Presiona Tab o Enter para confirmar

PASO 5: Agregar justificación (opcional pero recomendado)

- └ Haz clic en "Agregar nota"
- └ Escribe por qué cambiaste (ej: "Mayor espesor de pared")

PASO 6: Guardar cambios

- └ Botón "Guardar Cambios" abajo de tabla
- └ Sistema valida:
 - └ Valores numéricos válidos
 - └ No negativos
 - └ Precisión 4 decimales
- └ Respuesta:
 - └ ☒ "Cambios guardados exitosamente"
 - └ Sistema inserta en historial_cambios

PASO 7: Verificar historial

- └ Ir a Dashboard → Ver Historial
 - └ Confirmar que tu cambio aparece

Errores comunes:

Error	Causa	Solución
"Campo bloqueado"	Intentas editar APU1	Solo editar incidencia (APU2)
"Valor inválido"	Letras en campo numérico	Ingresar solo números
"No guardado"	No hiciste clic en Guardar	Presiona botón Guardar

Procedimiento 3: Normalizar Compras en Ajuste Manual (Especialista)

Tiempo: 2-4 horas para todos los insumos

Nivel: Especialista de presupuesto

PASO 1: Abrir /ajuste-manual

- └ Sistema carga tabla de insumos únicos

PASO 2: Buscar insumo (ej: "CEMENTO GRIS")

- └ Búsqueda booleana: AND obligatorio
- └ Ejemplo: "CEMENTO" AND "GRIS" AND "25KG"
- └ Retorna insumos coincidentes

PASO 3: Seleccionar un insumo de resultados

- └ Carga: todas sus compras
- └ Muestra tabla con:
 - └ Orden, Detalle
 - └ Unidad Original (bol, bolsa, saco)
 - └ Cantidad Original (500, 250, 100)
 - └ Unidad Normalizada (kg)
 - └ Cantidad Normalizada (editable)
 - └ Precio Normalizado (editable)
- └ Meta Global calculada automáticamente

PASO 4: Normalizar CADA compra

- └ Para cada fila:
 - └ Decide unidad estándar (ej: kg)
 - └ Ingresa cantidad en esa unidad
 - └ Ej: "1 bol CEMENTO 25kg" → 25 kg
 - └ Ingresa precio unitario normalizado
- └ Presiona Tab para confirmar

PASO 5: Validar suma

- └ Sistema muestra: Total Adquirido = $\Sigma(\text{cantidad_und})$
- └ Ejemplo: $500 + 250 + 100 = 850 \text{ kg}$
- └ Esto es la META de cuadro

PASO 6: Renombrar insumo (si es necesario)

- └ Si compras dicen "CEMENTO PORTLAND" pero presupuesto dice "CEMENTO GRIS"
- └ Haz clic en "Renombrar Insumo"
- └ Ingresa nombre oficial (ej: "CEMENTO GRIS PORTLAND")
- └ Se cambia en insumos + compras + historial

PASO 7: Editar APU2 por partida

```
└ Sistema muestra tabla de partidas que usan este insumo
└ Para CADA partida:
  └ Ve incidencia APU1 (referencia)
  └ Ve incidencia APU2 (editable)
  └ Edita cantidad_2 para cuadrar
  └ Ejemplo:
    └ META = 850 kg total
    └ 3 partidas usan CEMENTO
    └ Distribución: 300 + 350 + 200 = 850 ✓
    └ Balance = 0 (verde)
└ Presiona "Guardar APU"
```

PASO 8: Verificar cuadro final

```
└ Sistema muestra:
  └ Meta Global (850 kg)
  └ Total Adquirido (850 kg)
  └ Balance (0)
  └ Estado (● VERDE)
└ Si falta/exceso:
  └ Naranja: usuario debe justificar
  └ Guardar con comentario explicativo
```

PASO 9: Registrar cambios

```
└ Todos los pasos registrados automáticamente en historial_cambios
└ Consultable vía Dashboard → Historial
```

Tips:

- ✓ Normalizar PRIMERO todos los datos
- ✓ Luego editarp APU2
- ✓ Dejar notas si hay discrepancias
- ✓ No forzar cuadro si no tiene sentido



VALIDACIONES Y CONSTRAINTS

Validaciones a Nivel de Aplicación

Validación	Condición	Acción
Precisión Numérica	Valor > 4 decimales	Rechazar, mostrar error
Rango de Incidencia	Valor < 0	Rechazar, marcar rojo
FK - código_partida	No existe en partidas	Rechazar, sugerir correctos
FK - compra_id	No existe en compras	Rechazar, rollback

Validación	Condición	Acción
Duplicado Vinculación	mapeo_vinculacion (insumo, compra) duplicado	Rechazar, mostrar existente
Usuario Requerido	Header X-Usuario vacío	Rechazar, pedir login
JSON Válido	Request malformado	Error 400 Bad Request

Validaciones a Nivel de Base de Datos

```
-- PK: No duplicados
CONSTRAINT partidas_pk PRIMARY KEY (codigo)
CONSTRAINT compras_pk PRIMARY KEY (id)

-- FK: Integridad referencial
CONSTRAINT insumos_fk_partidas FOREIGN KEY (codigo_partida)
  REFERENCES partidas(codigo)
CONSTRAINT mapeo_fk_compras FOREIGN KEY (compra_id)
  REFERENCES compras(id) ON DELETE CASCADE

-- UNIQUE: Un insumo una sola vez por compra
CONSTRAINT mapeo_unique UNIQUE (compra_id)

-- CHECK: Lógica de negocio
CONSTRAINT insumos_positivos CHECK (cantidad_modificada >= 0)
CONSTRAINT compras_positivos CHECK (cantidad_und >= 0)

-- NOT NULL: Campos obligatorios
partidas.codigo NOT NULL
partidas.metrado_fijo NOT NULL
insumos.descripcion NOT NULL
compras.insumo_descripcion NOT NULL
```

Validaciones de Cuadre

Validación	Fórmula	Estado
Cuadre Exacto	ABS(Meta - Adquirido) < 0.0001	Verde
Falta Compra	Meta - Adquirido > 0.0001	Naranja
Exceso	Meta - Adquirido < -0.0001	Amarillo
Justificación Requerida	Balance != 0	Debe haber comentario

Modelo de Seguridad Actual

Limitación: No hay autenticación oficial, solo header **X-Usuario:**

```
// Backend detecta usuario pero NO lo valida contra BD
const usuario = req.headers['x-usuario'];
// Solo registra: "juan.garcia@obra.pe cambió X"
// NO verifica si existe o tiene permisos
```

Recomendaciones de Seguridad (Fase 2)

1. AGREGAR TABLA: users
- └ id, email, nombre, rol, activo
- └ Roles: ADMIN, ESPECIALISTA, AUDITOR, VISUALIZADOR
2. IMPLEMENTAR AUTENTICACIÓN
- └ JWT o Session con httpOnly cookie
- └ Login en ruta protegida /auth/login
- └ Logout en /auth/logout
3. AGREGAR AUTORIZACIÓN
- └ ADMIN: Puede editar todo + eliminar usuarios
- └ ESPECIALISTA: Puede editar insumos/compras
- └ AUDITOR: Solo lectura + genera reportes
- └ VISUALIZADOR: Solo lectura dashboard
4. ENCRIPTAR DATOS SENSIBLES
- └ Credenciales BD en env variables
- └ Passwords con bcrypt (nunca plaintext)
- └ Conexión SSL/TLS a BD
5. AUDITORÍA MEJORADA
- └ Registrar IP address actual
- └ Registrar user agent (navegador)
- └ Alertas si cambios sospechosos
- └ Logs centralizados (no solo en BD)

Control de Acceso por Tabla (Propuesto)

Tabla	ADMIN	ESPECIALISTA	AUDITOR	VISUALIZADOR
partidas	R	R	R	R
insumos	R/W	R/W	R	R
compras	R/W	R/W	R	R

Tabla	ADMIN	ESPECIALISTA	AUDITOR	VISUALIZADOR
mapeo_vinculacion	R/W	R/W	R	R
historial_cambios	R/W/D	—	R	R
users	R/W/D	R	R	—



BACKUP Y RECUPERACIÓN

Estrategia de Backup

```
# Backup diario (automático vía cron)
#!/bin/bash
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
pg_dump -U postgres 7_insumos_rado \
  > /backups/7_insumos_rado_${TIMESTAMP}.sql

# Retención: últimos 30 días
find /backups -name "7_insumos_rado_*.sql" -mtime +30 -delete
```

Recuperación Ante Desastre

```
# Restaurar desde backup
psql -U postgres 7_insumos_rado < /backups/7_insumos_rado_20260525.sql

# Verificar integridad
psql -U postgres -d 7_insumos_rado -c "
  SELECT COUNT(*) as partidas FROM partidas;
  SELECT COUNT(*) as insumos FROM insumos;
  SELECT COUNT(*) as compras FROM compras;
"
```

Recuperación de Cambios Accidentales

Si un usuario edita por error:

```
-- 1. Ver qué cambió
SELECT * FROM historial_cambios
WHERE usuario = 'usuario@error.com'
AND fecha > '2026-05-25 10:00:00'
ORDER BY fecha DESC;
```

```
-- 2. Crear cambio inverso (NO borrar el original)
INSERT INTO insumos (codigo_partida, descripcion, incidencia, ...)
VALUES (...) -- valores originales
-- Agregar nota en historial: "Revertido por corrección"

-- 3. Auditoría queda completa
SELECT * FROM historial_cambios
WHERE registro_desc LIKE '%corrección%';
```

PERFORMANCE Y OPTIMIZACIÓN

Índices Recomendados

```
-- Búsquedas frecuentes
CREATE INDEX idx_insumos_descripcion
ON insumos(descripcion);

CREATE INDEX idx_insumos_codigo_partida
ON insumos(codigo_partida);

CREATE INDEX idx_compras_insumo_descripcion
ON compras(insumo_descripcion);

-- Auditoría
CREATE INDEX idx_historial_usuario
ON historial_cambios(usuario, fecha DESC);

CREATE INDEX idx_historial_tabla_registro
ON historial_cambios(tabla, registro_id);

-- Vinculación
CREATE INDEX idx_mapeo_insumo_nombre
ON mapeo_vinculacion(insumo_nombre);

CREATE INDEX idx_mapeo_compra_id
ON mapeo_vinculacion(compra_id);
```

Benchmarks Esperados

Operación	Tiempo Objetivo	Carga
GET /api/partidas	< 100ms	1,134 registros
GET /api/compras?insumo=X	< 200ms	1,437 compras, búsqueda
POST /api/compras (guardar)	< 500ms	Transacción + auditoría

Operación	Tiempo Objetivo	Carga
Exportar Excel	< 3s	Generación de 4 hojas
Búsqueda booleana	< 300ms	Index en descripción

Optimizaciones

```
// 1. Pool de conexiones (ya implementado)
const pool = new Pool({ max: 20, min: 5 });

// 2. Queries paralelas (si es posible)
const [partidas, insumos] = await Promise.all([
  pool.query('SELECT * FROM partidas'),
  pool.query('SELECT * FROM insumos')
]);

// 3. Paginación (para tablas grandes)
GET /api/compras?page=1&limit=50
// Retorna 50 en vez de 1,437

// 4. Caching de vistas (si hay hits repetidos)
GET /api/apu-full?cache=true&ttl=3600
// Cache 1 hora si no hay cambios
```



ROLES Y RESPONSABILIDADES

Rol 1: Administrador del Sistema

Responsabilidades:

- ✓ Instalación y configuración inicial
- ✓ Creación de usuarios
- ✓ Backups y recuperación
- ✓ Mantenimiento de BD
- ✓ Reportes de performance

Permisos:

- CRUD en todas las tablas
- Acceso a logs de sistema
- Puede resetear credenciales

Número: 1-2 personas

Rol 2: Especialista de Presupuesto

Responsabilidades:

- ☒ Normalizar unidades de compras
- ☒ Editar APU2 (cantidades)
- ☒ Vincular compras a insumos
- ☒ Justificar discrepancias
- ☒ Exportar reportes

Permisos:

- Editar: insumos, compras, mapeo_vinculacion
- Leer: partidas, apus_detallado, historial_cambios

Número: 2-3 personas

Rol 3: Operador de Control

Responsabilidades:

- ☒ Editar incidencias en /control-insumos
- ☒ Ingresar datos de campo
- ☒ Registrar cambios con justificación
- ☒ Alertar sobre discrepancias

Permisos:

- Editar: incidencia, cantidad_modificada
- Leer: todas las tablas

Número: 3-5 personas

Rol 4: Auditor / Ente de Control

Responsabilidades:

- ☒ Revisar historial de cambios

- ☒ Validar cuadros
- ☒ Detectar anomalías
- ☒ Reportar a OSCE
- ☒ No pueden editar

Permisos:

- Lectura total (no escritura)
- Acceso a historial_cambios
- Exportación de reportes

Número: 1-2 personas



CAPACITACIÓN DE USUARIOS

Módulo 1: Inducción (30 min)

Contenido:

- Propósito del sistema
- Términos clave (APU1, APU2, cuadro)
- Estructura de BD
- Cómo reportar problemas

Formato: Video + documento

Módulo 2: /control-insumos (45 min)

Contenido:

- Qué es incidencia
- Cómo editar sin romper datos
- Cómo guardar cambios
- Cómo ver historial

Formato: Tutorial paso a paso + ejercicio práctico

Módulo 3: /ajuste-manual (60 min)

Contenido:

- Normalización de unidades
- Búsqueda booleana
- Cálculo de cuadros
- Justificación de discrepancias

Formato: Workshop en vivo + simulación de datos

Módulo 4: Auditoría y Reportes (30 min)

Contenido:

- Cómo leer historial_cambios
- Cómo generar reportes
- Cómo identificar anomalías

Formato: Demo en vivo

Evaluación

Cada usuario debe aprobar quiz:

- ☐ Definir APU1 vs APU2
 - ☐ Explicar qué es cuadro
 - ☐ Normalizar unidades (ejercicio)
 - ☐ Leer historial de cambios
-

? FAQ Y PREGUNTAS FRECUENTES

P: ¿Puedo borrar un cambio que hice por error?

R: No puedes BORRAR, pero puedes REVERTIR:

1. Ve a Dashboard → Historial
2. Encuentra tu cambio erróneo
3. Ingresa el valor original manualmente (nuevo cambio)

4. Agrega nota: "Revertido porque fue error"
5. Ahora el historial muestra: error → corrección

Beneficio: Auditoría 100% transparente

P: ¿Qué pasa si no cuadra un insumo?

R: Tres opciones:

A) **Falta compra** (Balance negativo):

- Buscar si fue comprado con otro nombre
- O justificar por qué no se ejecutó
- Editar APU2 a 0

B) **Exceso** (Balance positivo):

- Justificar ampliación
- O distribuir a más partidas

C) **No se puede justificar:**

- Reportar a auditor
 - Crear nota en historial
 - Dejar para investigación
-

P: ¿Puedo cambiar `metrado_fijo` de una partida?

R: NO, NUNCA.

`metrado_fijo` es del expediente técnico original. Si necesitas cambiar es porque la partida se ejecutó diferente → edita APU2 (incidencia), no `metrado_fijo`.

P: ¿Cómo veo quién cambió qué?

R: Dashboard → Historial de Cambios

Filtros:

- Por usuario
- Por tabla

- Por fecha
- Por insumo

P: ¿Puedo reventar el cuadro a propósito?

R: Técnicamente sí, pero:

- ✗ Se va a registrar en historial
- ✗ OSCE lo va a detectar
- ✗ Auditoría te va a preguntar
- ✗ Es fraude

No intentes. El sistema es para transparencia.

P: ¿Y si hay conflicto de ediciones (2 usuarios editando al mismo tiempo)?

R: Cada usuario edita por separado:

1. Usuario A edita incidencia de CEMENTO
2. Usuario B edita cantidad_und de OC-100
3. Ambas ediciones se guardan
4. Historial muestra ambas (con timestamps)
5. No hay conflicto porque son campos diferentes

Si editan EL MISMO CAMPO:

- Última edición gana (last-write-wins)
 - Se recomienda comunicación entre usuarios
-

P: ¿Qué pasa si PostgreSQL se cae?

R:

1. Sistema no funciona (BD está caída)
2. Operador nota que no carga datos
3. Avisa a administrador

4. Admin: a) Reinicia PostgreSQL b) O restaura desde backup más reciente c) Valida integridad
5. Sistema vuelve online
6. Cualquier cambio no guardado se pierde

Prevención: Hacer commits frecuentes (Guardar cada 15 min)

P: ¿Puedo exportar a formato diferente a Excel?

R: Actualmente solo Excel. Roadmap incluye:

- CSV (Fase 2)
 - PDF (Fase 2)
 - JSON API (Fase 2)
-

P: ¿Hay móvil?

R: No. Roadmap futuro pero no es prioritario.

- Sistema requiere tablas grandes
 - Mejor en desktop
 - Móvil podría ser para "lectura solo" (auditoría)
-



ROADMAP Y DESARROLLO FUTURO

Fase 1: MVP (ACTUAL - Completado)

-  Módulos 1 y 2 en producción
-  Auditoría básica
-  Exportación Excel
-  Header X-Usuario

Fase 2: Mejoras (Siguiendo - 2026-Q3)

Funcionalidad:

- ⌚ Módulo 3: Vinculador completo (UI visual)
- ⌚ Autenticación real (JWT/OAuth)
- ⌚ Roles y permisos en BD
- ⌚ Exportación CSV y PDF
- ⌚ Dashboard de KPIs
- ⌚ Alertas de discrepancias

Performance:

- ⌚ Caché en Redis
- ⌚ Índices adicionales
- ⌚ Paginación en tablas grandes
- ⌚ Lazy loading en frontend

Operación:

- ⌚ Importación de múltiples archivos Excel
- ⌚ Validación automática de unitarios
- ⌚ Sugerencias de vinculación (ML)
- ⌚ Reportes automáticos vía email

Fase 3: Integración (2026-Q4)

- ⌚ SIAF Integration (Sistema Financiero)
- ⌚ OSCE API (Entes de Control)
- ⌚ Almacén Integration (inventario)
- ⌚ Cloud Deployment (AWS/Azure)

Fase 4: Expansión (2027+)

- ⌚ Móvil (iOS/Android, solo lectura)
- ⌚ Multi-obra (gestionar varias proyectos)
- ⌚ Machine Learning (detección anomalías)
- ⌚ BI/Analytics (Data Warehouse)



MÉTRICAS Y KPIs

KPI 1: Tasa de Cuadre

Fórmula: $\text{Insumos con } |\text{Balance}| < 0.0001 / \text{Insumos Totales} \times 100$

Objetivo: > 95%

Actual: ~89% (a mejorar)

Cálculo:

Insumos Totales = 701

Cuadrados = 623

Tasa = $623/701 \times 100 = 88.8\%$

KPI 2: Tiempo Promedio de Normalización

Fórmula: $\text{Suma de (Fecha Cuadre - Fecha Carga)} / \# \text{ Insumos}$

Objetivo: < 5 días por insumo

Actual: Pending

Propósito: Medir velocidad de regularización

KPI 3: Discrepancia Total

Fórmula: $\text{Suma de ABS(Balance)} / \text{Meta Global} \times 100$

Objetivo: < 2%

Actual: Pending

Propósito: Ver cuánto "dinero" está sin cuadrar

KPI 4: Cambios por Usuario

Fórmula: $\# \text{ de cambios en historial_cambios por usuario}$

Propósito: Identificar quién está trabajando

Alerta: Usuario con cambios >1000 en 1 día = revisar

Ejemplo:

juan.garcia: 234 cambios (normal)

maria.lopez: 1,500 cambios en 2 horas (sospechoso)

KPI 5: Cobertura de Vinculos

Fórmula: $\text{Compras Vinculadas} / \text{Compras Totales} \times 100$

Objetivo: 100%

Actual: 73% (141 compras huérfanas)

Propósito: Asegurar que 100% de compras estén vinculadas



CONCLUSIÓN Y SIGUIENTES PASOS

Resumen del Documento

Este documento describe **de forma exhaustiva** el Sistema de Control y Ajuste de Insumos para la obra Belemppampa:

- ✓ **Propósito:** Regularizar y auditar obra pública con trazabilidad 100%
- ✓ **Tecnología:** Next.js + React + PostgreSQL + ExcelJS
- ✓ **Operación:** 4 tablas editable, 2 inmutables, auditoría completa
- ✓ **Módulos:** Control (editar incidencias) + Ajuste Manual (cuadre) + Vinculador (links)
- ✓ **Seguridad:** Audit trail, validaciones, constraints DB
- ✓ **Escalabilidad:** Soporta 10K insumos, 50 usuarios simultáneos

Próximos Pasos Inmediatos

Week 1: Preparación

- [] Revisar documento con equipo
- [] Validar interpretación correcta
- [] Hacer preguntas/ajustes
- [] Acordar fecha de go-live

Week 2-3: Carga Inicial

- [] Cargar partidas desde ACU_Acumulado.xlsx
- [] Cargar compras desde DATA_INSUMOS.xlsx
- [] Cargar APU detallado desde APUS_Extraidos_v2.csv
- [] Validar integridad de datos
- [] Hacer backup inicial

Week 4: Capacitación

- [] Entrenar especialistas en /control-insumos
- [] Entrenar especialista en /ajuste-manual
- [] Entrenar auditor en historial/reportes
- [] Hacer pruebas E2E

Week 5: Go-Live

- [] Operación del sistema
- [] Monitoreo de performance
- [] Soporte a usuarios
- [] Comenzar regularización

Contactos y Escalación

Rol	Contacto	Teléfono	Email
Admin Sistema	TBD	—	—
Especialista Presupuesto	TBD	—	—
Auditor	TBD	—	—
Operador	TBD	—	—

Documentación Adicional

-  [SQL_Architecture_Master_Guide.md](#) — Ref. técnica detallada
-  [SISTEMA_BELEMPAMPA.md](#) — Descripción de módulos
-  [GUIA_SISTEMA_BELEMPAMPA.md](#) — Guía de usuario
-  [DEPLOYMENT_CHECKLIST.md](#) — Checklist instalación

Soporte y Mantenimiento

Horario de Soporte: Lunes-Viernes 8:00-17:00
Tickets: issues@obra.belempampa.pe
Emergencias: +51-XXX-XXX-XXXX

Versiones y Cambios

Versión	Fecha	Cambios
1.0	2026-04-01	MVP inicial
2.0	2026-05-01	Auditoría mejorada + Vinculador beta
3.0	2026-05-25	Documentación completa (este documento)
3.1 (planeado)	2026-06-01	Módulo 3 completado
4.0 (planeado)	2026-08-01	Autenticación + Roles

Documento Preparado Por: Sistema Automatizado de Documentación

Última Actualización: 25 de mayo de 2026, 14:30 UTC

Versión: 3.0 — Documentación Completa y Expandida

Estado:  **LISTO PARA PRODUCCIÓN**

Aprobado Por: Equipo Técnico (Pendiente firma)

Notas Finales

Este documento está vivo. Debe actualizarse cuando:

-  Se lanzan nuevas features
-  Se descubren bugs importantes
-  Cambian procesos operacionales
-  Hay cambios de personal/roles

Responsable de Actualización: Administrador del Sistema

Frecuencia Recomendada: Cada sprint (cada 2 semanas)

Almacenamiento: Repositorio Git + Wiki interna

 **¡Documentación Completada!**

Gracias por usar el Sistema de Control y Ajuste de Insumos.

Para soporte o preguntas, contacta al equipo administrativo.

Requisitos Previos

- PostgreSQL 14+ instalado
- Node.js 18+
- `next.js 16.2.4` + `react 19`
- Acceso a BD local

Pasos de Instalación

```
# 1. Clonar repositorio
git clone <repo-url>
cd 7_Insumos_rado

# 2. Instalar dependencias frontend
cd frontend
npm install

# 3. Configurar variables de entorno
# Crear .env con:
DB_USER=postgres
DB_PASSWORD=Jo.9839514500
DB_HOST=localhost
DB_PORT=5432
DB_NAME=7_insumos_rado

# 4. Crear schema en PostgreSQL
psql -U postgres -d 7_insumos_rado -f sql/00_CREATE_SCHEMA.sql

# 5. Cargar datos iniciales
python ../ingest_acu.py
python ../ingest_compras.py
python ../ingest_apus_to_pg.py

# 6. Iniciar servidor frontend
npm run dev
# Acceder a http://localhost:3000
```

Verificación de Deploy

```
# Health check
curl http://localhost:3000/api/partidas
# Debe retornar JSON válido

# Check DB connection
npm run test:db
```



TROUBLESHOOTING

Error: "Cannot connect to database"

Verificar:

1. PostgreSQL está corriendo: `systemctl status postgresql`
2. Credenciales en `.env` son correctas
3. DB existe: `psql -U postgres -l | grep 7_insumos_rado`
4. Puerto 5432 está abierto: `netstat -tuln | grep 5432`

Error: "Tabla no existe: partidas"

Ejecutar:

```
psql -U postgres -d 7_insumos_rado -f sql/00_CREATE_SCHEMA.sql
```

Error: "Foreign Key Constraint Violation"

Causas comunes:

1. Insumo con `codigo_partida` que no existe
→ Verificar: `SELECT * FROM insumos WHERE codigo_partida NOT IN (SELECT codigo FROM partidas);`
2. Compra con `compra_id` inválido
→ Verificar: `SELECT * FROM mapeo_vinculacion WHERE compra_id NOT IN (SELECT id FROM compras);`

Lentitud en búsqueda de insumos

Crear índices:

```
CREATE INDEX idx_insumos_descripcion ON insumos(descripcion);  
CREATE INDEX idx_compras_insumo_desc ON compras(insumo_descripcion);
```



NOTAS FINALES

Convenciones

- **Precisión:** Siempre 4 decimales (`NUMERIC(15,4)`)
- **Timestamps:** UTC (`TIMESTAMPTZ`)
- **Headers:** `X-Usuario` obligatorio en todas las mutaciones
- **Respuestas:** JSON con estructura `{ success, message, data }`

Pendiente de Implementación

-  Módulos 1 y 2 (Control + Ajuste Manual)
-  Módulo 3 (Vinculador) - UI en progreso
-  Integración con OSCE (exportación de reportes)

Referencia

- Proyecto previo: [Entregable_2_insumos_liquid](#) (Streamlit + Supabase)
 - Documentación SQL: [SQL_Architecture_Master_Guide.md](#)
 - Guía de uso: [GUIA_SISTEMA_BELEMPAMPA.md](#)
-

Documento Preparado Por: Sistema Automatizado

Último Update: 25 de mayo de 2026

Versión: 3.0 — Documentación Consolidada Completa